

## Стандартные библиотеки на GPU

# Библиотеки CUDA

<http://developer.nvidia.com/cuda-tools-ecosystem>

- CUDA Toolkit
  - Thrust : библиотека шаблонов
  - NPP: обработка изображений и сигналов
  - CURAND: псевдо- и квазислучайные числа
  - LIBM: стандартная библиотека математических примитивов
  - CUSPARSE : линейная алгебра для разреженных матриц
  - CUBLAS: линейная алгебра для плотных матриц
  - CUFFT: преобразование Фурье

# Библиотеки CUDA

- Библиотеки сторонних разработчиков
  - MAGMA: линейная алгебра
  - OpenVidia: компьютерное зрение
  - CULA Tools: линейная алгебра
  - OpenCurrent: динамика жидких сред
  - CUSP: решатели разреженных матриц
  - NAG: вычислительная финансовая система
  - ...

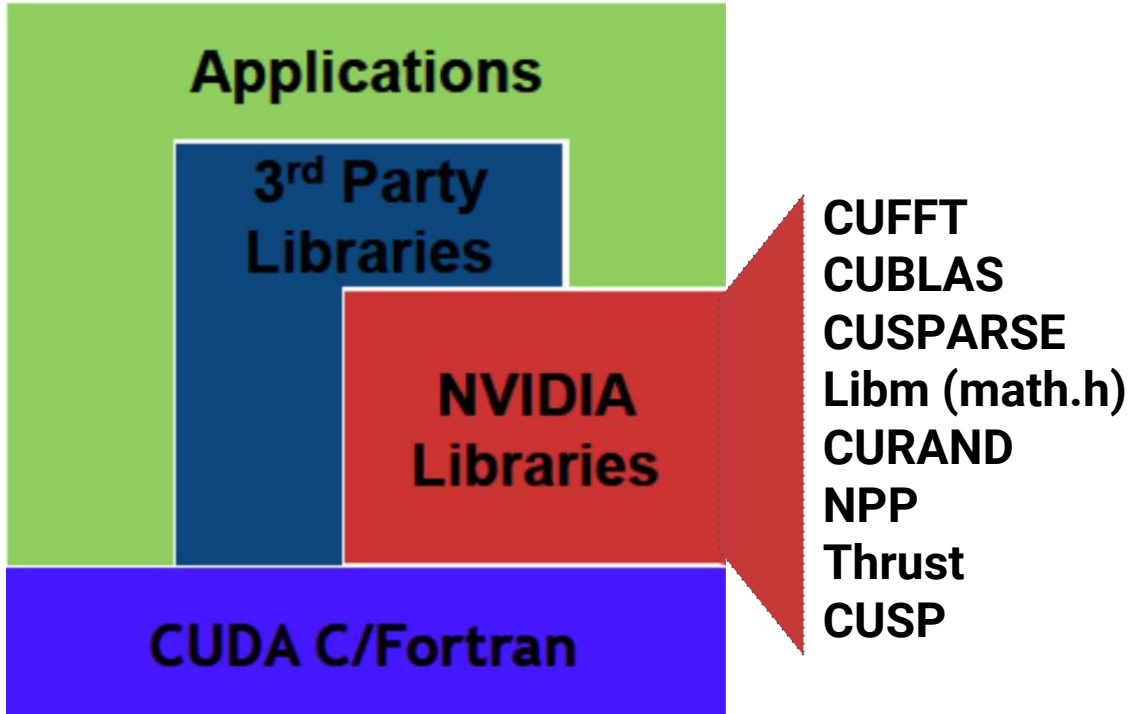
# Библиотеки

Специализированные параллельные библиотеки

([https://parallel.ru/tech/tech\\_dev/par\\_libs.html](https://parallel.ru/tech/tech_dev/par_libs.html))

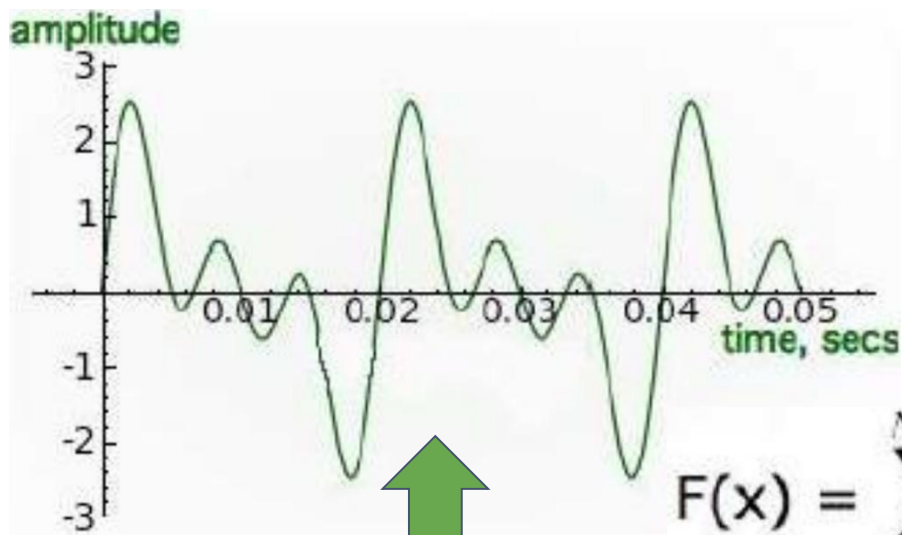
- ATLAS
- Aztec
- BlockSolve95
- Distributed Parallelization at CWP
- DOUG
- GALOPPS
- JOSTLE
- NAMD
- P-Sparslib
- PIM
- ParMETIS
- PARPACK
- PBLAS
- PETSc
- PGAPack
- PLAPACK, ScaLAPACK
- SPRNG

# Библиотеки

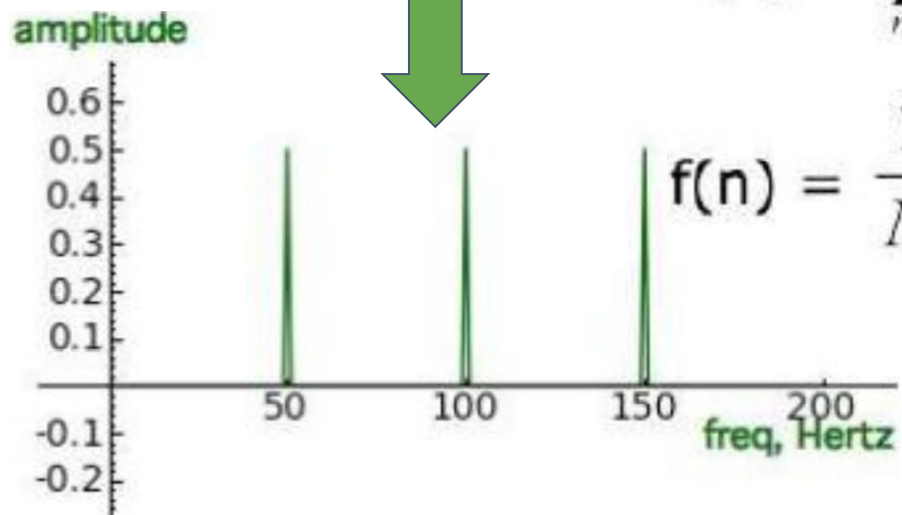


# CUFFT

Быстрое преобразование Фурье



$$F(x) = \sum_{n=0}^{N-1} f(n) e^{-j2\pi(x\frac{n}{N})}$$



$$f(n) = \frac{1}{N} \sum_{x=0}^{N-1} F(x) e^{j2\pi(x\frac{n}{N})}$$

# Особенности CUFFT

- Интерфейс подобен FFTW
- Одно-, двух- и трехмерные вещественные и комплексные преобразования
- Одинарная и двойная точность
- Одномерные преобразования до 128 миллионов элементов
- Потокое асинхронное вычисление

# Использование CUFFT

- Шаг 1: выделить память на GPU
- Шаг 2: создать и настроить преобразование (размер, тип...)
- Шаг 3: выполнить преобразование столько раз, сколько необходимо, используя указатель из шага 1
- Шаг 4: уничтожить преобразование и освободить память на GPU



# Вызов CUFFT

```
#define NX 256
#define NY 128

cufftHandle plan;
cufftComplex *idata, *odata;
cudaMalloc((void**)&idata, sizeof(cufftComplex)*NX*NY);
cudaMalloc((void**)&odata, sizeof(cufftComplex)*NX*NY);

...
/* Create a 2D FFT plan. */
cufftPlan2d(&plan, NX, NY, CUFFT_C2C);

/* Use the CUFFT plan to transform the signal out of place_ */
cufftExecC2C(plan, idata, odata, CUFFT_FORWARD);

/* Inverse transform the signal in place_ Different pointers to
input and output arrays implies out of place transformation */
cufftExecC2C(plan, odata, odata, CUFFT_INVERSE);

...
/* Destroy the CUFFT plan_ */
cufftDestroy(plan);

cudaFree(idata), cudaFree(odata);
```

# CUBLAS

- Реализация BLAS
  - Basic Linear Algebra Subprograms
- Поддержка всех функций BLAS
  - Level1 (вектор-вектор):  $O(N)$ 
    - AXPY:  $y = \alpha \cdot x + y$
    - DOT:  $\text{dot} = x \cdot y$
  - Level2 (матрица-вектор):  $O(N^2)$ 
    - Vector multiplication by a General Matrix : GEMV
    - Triangular solver : TRSV
  - Level3 (матрица-матрица):  $O(N^3)$ 
    - General Matrix Multiplication : GEMM
    - Triangular Solver : TRSM

# Особенности CUBLAS

- Поддержка 4 типов:
  - Float, Double, Complex, Double Complex
  - Префиксы: S, D, C, Z
- 152 функции: S(37), D(37), C(41), Z(41)
- Название функций: cublas + BLAS\_name
- Пример: cublasSGEMM
  - S: одинарной точности
  - GE: general (общего вида)
  - M: multiplication (умножение)
  - M: matrix (матрица)

# CURAND

- Генерация псевдослучайных чисел
- Возможность API вызов на хосте для генерации случайных чисел
- Возможность использовать внутри GPU функций/ядер
- Равномерные, нормальные и логнормальные распределения одинарной и двойной точности

# Использование CURAND

- Создание генератора  
`curandCreateGenerator()`
- Инициализация генератора  
`curandSetPseudoRandomGeneratorSeed()`
- Генерация данных из распределений  
Равномерное:  
`curandGenerateUniform()/curandGenerateUniformDouble()`  
Гауссово:  
`curandGenerateNormal()/cuRandGenerateNormalDouble()`  
Лог-нормальное:  
`curandGenerateLogNormal/curandGenerateLogNormalDouble()`
- Уничтожение генератора  
`curandDestroyGenerator()`

# Вызов CURAND

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda.h>
#include <curand.h>

main() {
    int i, n = 100;
    curandGenerator_t gen;
    float *devData, *hostData;

    /* Allocate n floats on host */
    hostData = (float *)calloc(n, sizeof(float));

    /* Allocate n floats on device */
    cudaMalloc((void **)&devData, n * sizeof(float));

    /* Create pseudo-random number generator */
    curandCreateGenerator(&gen, CURAND_RNG_PSEUDO_DEFAULT);
```

# Вызов CURAND

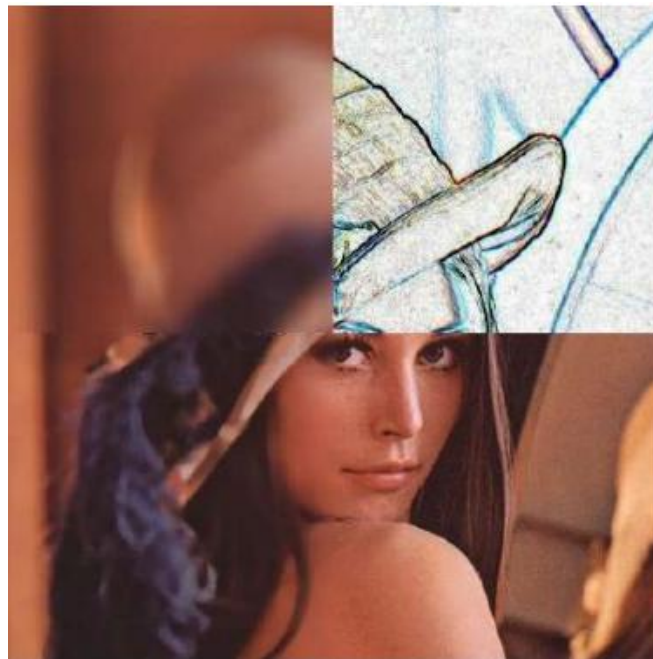
```
/* Set seed */
curandSetPseudoRandomGeneratorSeed(gen, 1234ULL);
/* Generate n floats on device */
curandGenerateUniform(gen, devData, n);
/* Copy device memory to host */
cudaMemcpy(hostData, devData, n * sizeof(float),
           cudaMemcpyDeviceToHost);

/* Show result */
for(i = 0; i < n; i++){
    printf("%1.4f ", hostData[i];
}
printf("\n");

/* Cleanup */
curandDestroyGenerator(gen);
cudaFree(devData);
free(hostData);

return 0;
}
```

- NVIDIA Performance Primitives
- Библиотека функций
  - Оптимизирована
  - Низкоуровневая
  - Исполняется на GPU
- ~350 функций для обработки изображений
- ~100 функций для обработки сигналов





# Thrust



- Библиотека шаблонов для CUDA
  - Похожа на C++ STL
- Контейнеры
  - Управление памятью на хосте и устройстве:
    - `thrust::host_vector`,
    - `thrust::device_vector`
  - Помогает избежать распространенные ошибки
- Итераторы
  - Знают, где находятся данные
- Алгоритмы
  - Сортировка
  - редукция
  - префиксная сумма
  - ...

# Алгоритмы

- Редукция
- Префиксная сумма (scan)
- Битоническая сортировка
- Свертка
- Построение гистограмм
- Поразрядная сортировка
- ...

## Стандартные библиотеки на GPU